

Machine Learning Project Report

1. Library Import

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.model_selection import train_test_split, GridSearchCV

from sklearn.ensemble import RandomForestClassifier
from sklearn.feature_selection import SelectFromModel

from sklearn.decomposition import PCA

from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix, f1_score, recall_score

from xgboost import XGBClassifier
```

Explanation:

In this step, the required libraries are imported to handle data processing, model building, evaluation, and visualization:

- pandas: for loading and manipulating tabular data
- numpy: for numerical operations
- matplotlib and seaborn: for data visualization
- sklearn: for machine learning preprocessing, modeling, and evaluation
- xgboost: for building a high-performance gradient boosting model

2. Load Dataset

```
df = pd.read_csv('you20.csv')
print(df.head())
print(df.info())
```

Explanation:

The dataset is loaded from a CSV file into a DataFrame. Then:

- head() displays the first rows to understand structure

- info() shows data types, missing values, and general dataset structure
-

3. Data Cleaning

```
df.replace("unknown", np.nan, inplace=True)

num_cols = df.select_dtypes(include=['int64', 'float64']).columns
cat_cols = df.select_dtypes(include=['object']).columns

for col in num_cols:
    df[col] = df[col].fillna(df[col].median())

for col in cat_cols:
    df[col] = df[col].fillna(df[col].mode()[0])
```

Explanation:

Data cleaning is a crucial step before training:

- "unknown" values are replaced with NaN
 - Numerical columns: missing values are filled with median (robust to outliers)
 - Categorical columns: missing values are filled with mode (most frequent value)
-

4. Convert Categorical Data to Numbers (Label Encoding)

```
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()
for col in df.select_dtypes(include='object').columns:
    df[col] = le.fit_transform(df[col])
```

Explanation:

Machine learning models cannot process text data directly, so:

- LabelEncoder converts each category into a unique numeric value
 - Example: red, blue, green → 0, 1, 2
 - This makes categorical data usable for ML algorithms
-

5. Split Features and Target

```
X = df.drop('y', axis=1)
y = df['y']
```

Explanation:

- X contains input features used for prediction
 - y is the target variable to be predicted
-

6. Train-Test Split

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

Explanation:

The dataset is split into:

- 80% training data
 - 20% testing data This ensures proper evaluation of model performance on unseen data.
-

7. Feature Scaling

```
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Explanation:

Standardization ensures all features are on the same scale:

- Mean = 0
 - Standard deviation = 1 This improves performance for many ML algorithms.
-

8. Random Forest Model

```
model = RandomForestClassifier(
    n_estimators=400,
```

```

        max_depth=25,
        random_state=42,
        min_samples_split=42,
        min_samples_leaf=1,
        class_weight="balanced"
    )

    model.fit(X_train_scaled, y_train)
    y_pred = model.predict(X_test_scaled)

    acc0 = accuracy_score(y_test, y_pred)
    print("Random Forest Accuracy:", acc0)

```

Explanation:

Random Forest is an ensemble model based on multiple decision trees:

- Each tree learns different patterns
- Final prediction is based on majority voting
- Reduces overfitting compared to a single decision tree
- `class_weight` helps handle imbalanced datasets

9. Feature Importance

```

importances = model.feature_importances_

feat_imp = pd.DataFrame({
    'Feature': X_train.columns,
    'Importance': importances
}).sort_values(by='Importance', ascending=False)

```

Explanation:

This step identifies which features contribute most to the model decision:

- Higher importance means stronger influence on prediction
- Useful for feature selection and interpretation

10. Feature Selection

```

selector = SelectFromModel(model, threshold='median')

X_train_selected = selector.fit_transform(X_train, y_train)
X_test_selected = selector.transform(X_test)

```

```
model.fit(X_train_selected, y_train)
y_pred = model.predict(X_test_selected)

acc = accuracy_score(y_test, y_pred)
```

Explanation:

- Keeps only the most important features
 - Reduces model complexity
 - May improve performance and reduce overfitting
-

11. PCA (Dimensionality Reduction)

```
for k in [5, 8, 10, 12, 14]:
    pca = PCA(n_components=k)

    X_train_pca = pca.fit_transform(X_train)
    X_test_pca = pca.transform(X_test)

    model.fit(X_train_pca, y_train)
    y_pred = model.predict(X_test_pca)

    acc2 = accuracy_score(y_test, y_pred)
```

Explanation:

- PCA reduces the number of features while preserving important information
 - Helps remove noise and redundancy
 - Different values of components are tested to find the best configuration
-

12. Model Evaluation

```
print(classification_report(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))
```

Explanation:

- Classification report includes precision, recall, and F1-score
 - Confusion matrix shows correct and incorrect predictions
 - Provides detailed model performance analysis
-

13. Visualization of Results

```
labels = ['RF', 'RF + FS', 'RF + PCA']
values = [acc0, acc, acc2]

plt.bar(labels, values)
plt.title("Model Accuracy Comparison")
plt.show()
```

Explanation:

Visual comparison of different model configurations based on accuracy.

14. Grid Search

```
grid = {
    'n_estimators': [50, 150, 200],
    'max_depth': [5, 10, 15]
}
```

Explanation:

Grid Search is used to find the best hyperparameters by testing multiple combinations.

15. XGBoost Model

```
model_xgb = XGBClassifier(
    n_estimators=500,
    max_depth=8,
    learning_rate=0.05,
    subsample=0.8,
    colsample_bytree=0.8,
    scale_pos_weight=1,
    gamma=1,
    random_state=42,
    eval_metric='logloss'
)
```

Explanation:

XGBoost is a powerful gradient boosting algorithm:

- Builds models sequentially to correct previous errors
- Highly efficient and widely used in competitions

- Handles complex patterns effectively

16. XGBoost + PCA (Dimensionality Reduction)

```
from sklearn.decomposition import PCA

pca = PCA(n_components=0.95)

X_train_pca = pca.fit_transform(X_train_scaled)
X_test_pca = pca.transform(X_test_scaled)

model_xgb_pca = XGBClassifier(
    n_estimators=500,
    max_depth=8,
    learning_rate=0.05,
    subsample=0.8,
    colsample_bytree=0.8,
    scale_pos_weight=1,
    gamma=1,
    random_state=42,
    eval_metric='logloss'
)

model_xgb_pca.fit(X_train_pca, y_train)

pred_xgb_pca = model_xgb_pca.predict(X_test_pca)

acc_xgb_pca = accuracy_score(y_test, pred_xgb_pca)
recall_xgb_pca = recall_score(y_test, pred_xgb_pca, average='weighted')
f1_xgb_pca = f1_score(y_test, pred_xgb_pca, average='weighted')

print("PCA + XGBoost Accuracy:", acc_xgb_pca)
print("Recall:", recall_xgb_pca)
print("F1 Score:", f1_xgb_pca)

print("
Classification Report:
", classification_report(y_test, pred_xgb_pca))
print("
Confusion Matrix:
", confusion_matrix(y_test, pred_xgb_pca))
```

Explanation:

- PCA reduces feature dimensions while keeping 95% of the variance
- This improves computational efficiency and reduces noise
- XGBoost is then trained on the reduced feature space
- Performance is evaluated using Accuracy, Recall, F1-score, and Confusion Matrix

17. Model Comparison (XGBoost)

```
import matplotlib.pyplot as plt

labels = ['XGBoost', 'XGBoost + PCA']
values = [acc_xgb, acc_xgb_pca]

plt.bar(labels, values)
plt.title("XGBoost Accuracy Comparison")
plt.ylabel("Accuracy")

for i, v in enumerate(values):
    plt.text(i, v, str(round(v, 4)), ha='center', va='bottom')

plt.show()

# F1 Score Comparison
f1_values = [f1_xgb, f1_xgb_pca]

plt.bar(labels, f1_values)
plt.title("XGBoost F1-Score Comparison")
plt.ylabel("F1 Score")

for i, v in enumerate(f1_values):
    plt.text(i, v, str(round(v, 4)), ha='center', va='bottom')

plt.show()
```

Explanation:

- This section compares model performance before and after applying PCA
- Accuracy and F1-score are used as evaluation metrics
- Visualization helps in understanding which configuration performs better

18. Final Conclusion

Explanation:

- Random Forest provides a strong baseline model for classification tasks
- Feature Selection reduces complexity and helps in focusing on important variables
- PCA reduces dimensionality and improves efficiency, but may affect accuracy depending on the dataset
- XGBoost generally achieves the best performance due to its advanced boosting mechanism
- The combination of PCA with XGBoost improves speed and may help in high-dimensional datasets